Document Number: P3311R0
Date: 2024-05-22
Audience: SG21
Reply-to: Tom Honermann <tom@honermann.net>

An opt-in approach for integration of traditional assert facilities in C++ contracts

Introduction

SG21 attained strong concensus to forward P2900R6 (Contracts for C++) [P2900R6] to EWG during the 2024-02-29 SG21 telecon as its design for a minimally viable solution for contracts in C++. This design does not integrate the traditional C assert facility, nor does it expose interfaces to enable other traditional or ad hoc assert facilities to integrate with the new contracts features.

The P2900R6 design includes a new `contract_assert` statement that avoids many of the well-known pitfalls inherent in macro based traditional assert facilities. Despite considerable desire and effort to enable use of the `assert` identifier in the contracts design, no method to do so was identified that both avoided backward compatibility problems and used a syntax that gained consensus. P2884R0 (`assert` Should Be A Keyword In C++26) [P2884R0] attempted to promote `assert` to a keyword, but failed to get consensus. P2961R2 (A natural syntax for Contracts) [P2961R2], was approved by SG21 during the Kona, 2023 meeting. Section 5.2, "The `assert` name clash", details the challenges faced with trying to repurpose that identifier.

As a core language feature, `contract_assert` does not provoke ODR violations due to `NDEBUG` being inconsistently defined across translation units, is not subject to being redefined during translation as is permissible for macros (note that `contract_assert` is proposed as a keyword and is therefore not available for use as an identifier), and is not confused by the presence of a comma that is not contained within a pair of balanced parenthesis (an issue addressed for C assert by P2264R7 (Make assert() macro user friendly for C and C++) [P2264R7] for C++26). `contract_assert` also guards against some cases of possibly unintended behavior that may otherwise occur when its associated *conditional-expression* is evaluated. It guards against possibly unintended mutation of some objects by implicitly applying a const qualifier to the result of *id-expression*s that name a parameter or variable with automatic storage duration and to the pointee type of *this* expressions. It guards against unintended dependence on side effects by allowing an implementation to elide evaluation when the result of the evaluation can be predetermined or to perform the evaluation multiple times (with an expectation that multiple evaluations will make such side effects more noticeable).

The differences between `contract_assert` and traditional assert facilities make it impossible to directly reimplement traditional assert facilities using `contract_assert` as currently proposed without imposing potentially extensive changes to code that uses those facilities. Further, the latter two behaviors (implicit application of a const qualifier and evaluation elision or duplication) have become contentious due to a lack of confidence that `contract_assert`, as currently proposed, suffices as a modern alternative to traditional assert facilities like C assert.

Proposed are extensions to the P2900R6 design that are intended to enable existing assert facilities, including C assert, to be optionally integrated with C++ contracts, with minimal changes, such that assertion failures that occur through use of those facilities are also reported through the C++ contract violation handler. In doing so, the author hopes to reduce the current contention over the design of `contract_assert` by allowing those that prefer the traditional evaluation and side effect guarantees to use interfaces that behave as they desire without compromising on the protections that `contract_assert`, as currently proposed, provides.

Motivation

Vast amounts of code have been written over the lifetime of the C and C++ languages that utilize C assert or other traditional or ad hoc assertion facilities. It is not realistic to expect or require existing uses of traditional assert facilities to be rewritten to use C++ contracts when such code is migrated to C++26. The limitations and pitfalls of traditional assert facilities provides good motivation for a new modern facility. However, greater motivation for use of a new facility is produced if it enables integration with existing facilities (such integration also helps to reduce the negative consequences of the infamous "there are N+1 competing standards" situation). Specifically, the consequences of use of multiple assertion facilities, a persistent reality for many programmers, is reduced if assertion failures can be handled in a uniform way.

While it is not feasible to unconditionally claim the `assert` identifier for use with C++ contracts without unacceptable backward compatibility impact, it is feasible to offer the ability for projects to opt-in to a mode that handles assertion failures in traditional `assert()` expressions by invoking the C++ contract violation handler. In the long term, perhaps with assistance from code modernization tools, such integration could become standard practice and eventually allow `assert` to be more fully integrated into C++ contracts.

Design considerations

Invoking the contract violation handler

There are two basic approaches to enable a traditional assert facility to invoke the contract violation handler.

- Expose an interface to call `::handle_contract_violation()` with a user constructed `std::contracts::contract_violation` object, or
- Expose a core language feature that evaluates an arbitrary expression and then calls `::handle_contract_violation()` with an appropriately constructed `std::contracts::contract_violation` object if that evaluation yields a false result.

Enabling user construction of `std::contracts::contract_violation` objects would require exposing a suitable constructor or factory function to specify property values to be returned by the various property access member functions. `std::contracts::contract_violation` is designed to allow these property values to be accessed from statically allocated memory associated with each contract assertion present in the program that has a potentially checked semantic. Support for user construction would require specifying an ownership model for user provided values. Further, it is currently implementation-defined whether the class is polymorphic; it could be an abstract class, but the property access member functions are non-virtual. Enabling user construction could provide flexibility for integration of traditional assert facilities with C++ contracts, but would also impose

responsibility for appropriate selection of the `contract_kind`, `contract_semantic`, and `detection_mode` property values.

`contract_assert` is a core language feature that is already capable of evaluating an expression and invoking the contract violation handler. The inability to use it to directly implement a traditional assert interface is superimposed by the express desire to avoid well known problems with those traditional interfaces. However, it is possible to relax those imposed restrictions.

The grammar for the `contract_assert` statement was designed with future extension in mind.

> *assertion-statement :*
> `contract_assert` *attribute-specifier-seq*$_{opt}$ `(` *conditional-expression* `) ;`

While use of an attribute to opt-in to alternative semantics for the `contract_assert` statement would not require syntactic extension, such use would violate the WG21 ignorability policy for standard attributes. The ignorability policy prohibits a standard attribute from selecting a behavior that is not one of the permissible behaviors an implementation would be allowed to select in the absence of the attribute; an attribute cannot be used to relax constraints.

Fortunately, there is room for context sensitive keywords to be used in between the `contract_assert` keyword and the opening parenthesis (either before or after the optional *attribute-specifier-seq*) or between the closing parenthesis and the semicolon.

Interaction with `NDEBUG`

C assert supports two translation modes. When the `NDEBUG` macro is defined, arguments to `assert` expressions are not evaluated; this is the traditional form of the *ignore* semantic (and a form that leads to ODR violations). When `NDEBUG` is not defined, `assert` exhibits behavior like that of the *enforce* semantic, but with more stringent requirements on how termination is performed; `abort()` is called and any further processing requires the installation of a signal handler for the `SIGABRT` signal.

Since `NDEBUG` may be used to conditionally remove code that an `assert` expression depends on, it isn't possible, in general, to parse arguments passed to `assert` when `NDEBUG` is defined. It is therefore impossible to use `NDEBUG` as a proxy for a contract semantic; when `NDEBUG` is defined, no contract specification is present. However, when `NDEBUG` is not defined, then all contract semantics are applicable, though maintaining conformance with C standard requirements for the behavior of `assert` requires selecting the *enforce* semantic, writing the required information to the standard error stream, and calling `abort()`.

Proposal

Extend traditional semantics to `contract_assert`

Add a new `traditional_assert` enumerator to `std::contracts::contract_kind`.

Extend the `contract_assert` syntax to allow an optional context sensitive `traditional` keyword after `contract_assert`.

> *assertion-statement :*
> `contract_assert` `traditional`$_{opt}$ *attribute-specifier-seq*$_{opt}$ `(` *conditional-expression* `) ;`

Modify the `contract_assert` behavior as follows when `traditional` is specified.

- Do not implicitly apply const qualification to parameters, variables with automatic storage duration, or the pointee type of `this`.
- Evaluate the *conditional-expression* exactly one time each time control reaches the `contract_assert` statement irregardless of implementation-defined selection of contract semantics.
- Propagate exceptions thrown during evaluation of the *conditional-expression*; do not catch them and invoke the violation handler.
- If the *conditional-expression* evaluation yields a false result, invoke the violation handler with a reference to a `std::contracts::contract_violation` object for which its `detection_mode()` member will return `predicate_false`, its `kind()` member will return `traditional_assert`, and its `semantic()` member will return `enforce`.

Extend the recommended practice for the default contract violation handler to implement the behavior required by the C standard for a failed assertion when invoked with a `contract_violation` object whose `kind()` member returns `traditional_assert`. This includes writing select information to the standard error stream and then calling `abort()`.

Note that the above calls for the violation handler to be invoked with the *enforce* contract semantic in effect. This is required because there is no mechanism for the `contract_assert` statement to indicate whether the *conditional-expression* was evaluated, what its result was, or whether the violation handler was invoked and returned. Code following the `contract_assert` statement therefore has no information available to determine what action should be taken next. This limitation can be worked around by evaluating the assertion predicate separately and using the result value for the *conditional-expression* of the `contract_assert` statement, but this effectively hides the expression from the contracts implementation and prevents exposing it via the `comment()` member of `std::contracts::contract_violation`. Should motivation arise, additional extensions to enable use of an alternative contract semantic can be added at a later time. Selection of an alternate semantic could be enabled by parameterizing `traditional` with an argument enclosed in angle brackets (e.g., `traditional<observe>`; use of parenthesis would be ambiguous due to the parenthesis that surround the *conditional-expression*).

Enable optional C++ contracts integration for C assert

Upon inclusion of the `assert.h` or `cassert` header, if the `NDEBUG` macro is not defined and the `ASSERT_USES_CONTRACTS` macro is defined, define the `assert` macro as follows.

```
#define assert(...) \
  [&] { contract_assert traditional (__VA_ARGS__); }()
```

Note that it is not an error to define both `NDEBUG` and `ASSERT_USES_CONTRACTS`; when `NDEBUG` is defined, `ASSERT_USES_CONTRACTS` has no effect. The effects of the `NDEBUG` macro must take precedence to preserve backward compatibility with existing code.

Implementation experience

None.

Acknowledgements

Thanks to all SG21 participants for the tremendous amount of time, energy, and expertise that has been contributed towards advancing contracts for C++.

References

[P2264R7] "Make assert() macro user friendly for C and C++", P2264R7, 2023.
https://wg21.link/p2264r7

[P2884R0] "`assert` Should Be A Keyword In C++26", P2884R0, 2023.
https://wg21.link/p2884r0

[P2900R6] "Contracts for C++", P2900R6, 2024.
https://wg21.link/p2900r6

[P2961R2] "A natural syntax for Contracts", P2961R2, 2023.
https://wg21.link/p2961r2

Wording

These changes are relative to P2900R6 [P2900R6].

☐ Hide inserted text
☐ Hide deleted text

Wording will be provided in a future revision contingent on SG21 encouraging further work for the proposed direction.